Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 — Unit 3: Functions, Functional Programming and Modules · Complete Unit Master

Notes-first study master · source-cited lecture explanations and compact page takeaways

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the edited lesson notes by topic. Evidence chips point to transcript intervals and slide pages. The complete source transcripts, page text, formula candidates, and image locators are preserved in sibling JSON datasets. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_03: Unit 3: Functions, Functional Programming and Modules
2. 2 · Recursion
3. 3 · Callback
4. 4 · Functional and object oriented programming
5. 5 · Lambda Function
6. 6 · List Comprehension
7. 7 · Closure
8. 8 · Decorators
9. 9 · Generators
10. 10 · module
11. 11 · Numpy
12. 12 · PIL
13. 13 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda
14. 14 · Source slide deck: UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda
15. 15 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce
16. 16 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip
17. 17 · Source slide deck: Max Min
18. 18 · Source slide deck: Intro to Modules
19. 19 · Source slide deck: Matplotlib
20. 20 · Source slide deck: argparse
21. 21 · Historical question-source pages · unit mapping candidates
22. Appendix A · Symbol glossary
23. Appendix B · Sources, coverage and gaps
24. Appendix C · Revision and retrieval practice

M1

Unit_03 · Unit 3: Functions, Functional Programming and Modules

1 UE26CS151A · Unit_03: Unit 3: Functions, Functional Programming and Modules

This unit master organizes source material for Python for computational Problem Solving, Unit_03, officially titled Unit 3: Functions, Functional Programming and Modules. It includes 11 video transcripts and 18 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review. EV0000001

SHORT-NOTE RECAP

Unit map · UE26CS151A · Unit_03: Unit 3: Functions, Functional Programming and Modules

Official catalogue entry: UE26CS151A, Unit_03 (Unit 3: Functions, Functional Programming and Modules). Bucket folder present: yes. Videos: 11; slide decks: 18.

EV0000001

2 Recursion

Study focus: Recursion. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000002 · EV0000003

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we are going to be looking into the concept called as recursion.

Key details

- Recursion is basically a process where a function calls itself in order to solve a problem.
- It's called.
- This is called as a recursion.

- There are few recursive functions or sometimes recursive functions are so useful that they make it very easy for us to write a code.
- And there are few algorithms where it is very easy to express in terms of a recursive paradigm rather than writing it as an iterative process.
- But make sure that whatever is required and the best approach in order to solve your requirement, make use of it.
- In order to write a recursive program, I need a recursive case.
- I say recursive case.
- I have to mention what is my recursive case.
- At the same time, I also need to specify something called as a base case.
- Hopefully don't mind with my spelling mistakes, base case, or I can also call it as a termination case.
- Also called as a termination case.
- What will happen if I do not have a base case?
- If you do not mention where this particular function should stop executing, if you do not mention the function gets called again and again and you end up in an infinite loop.
- That's the reason why whenever you are writing a recursive function, make sure you write a recursive case at the same time, a very important one which is called as a base case.
- The main advantages of recursion is it is simple in nature.
- And it is simple in nature in the sense the code looks so simple.
- Code looks simple.
- At the same time it is the moment it is simple in nature, it also looks elegant.
- And the second advantage is basically your task can be broken down into a simpler subtlask.
- Task is broken down.
- [ASR low confidence; check audio].
- The set task.
- Is broken down into subtask.
- These are the advantages of your recursion, and the disadvantages are.
- Let me just say this as advantage so it is easy for us to go through.
- And what are the disadvantages?
- The disadvantages are.
- Lots of memory.
- It consumes lots of memory.
- It becomes inefficient.
- At the same time, it also consumes lots of time.
- And if you do a mistake, very tough to debug.
- Because you never know where the mistake is happening.
- And understanding the logic behind the recursion is not so easy.
- Let me quickly iterate over the advantages.
- Code looks simple and elegant.
- Task is broken down into the subtasks.
- Disadvantages is it takes lots of time, at the same time it also consumes lots of memory.
- And programs written using recursion are very hard to debug and understanding the logic behind recursive program is not easy.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000002 · EV0000003 · EV0000004 · +1 source units in pack.json

Core explanation · 05:00–09:59

Very famous problem we usually write.

Key details

- What is this what do I want to find out?
- I want to find out what is a factorial of a particular number.
- I will say if the number that I give you if it is equal to zero, which means you want to find a factorial of zero.
- Because factorial of zero is one.
- Otherwise, what I will say else, I will also say it as what?
- Instead of doing that, perform X into fact of X minus one.
- Every time you're trying to find out the factorial, what I'm trying to basically do, I am trying to find the value and the previous one, x minus one.
- I have to return this, so I will say it as what?
- factorial x into factorial of x minus one.
- My function is ready.
- What's the next thing that I have to do?
- I have to call this function.
- You will do the every time okay, maybe we will take a simple one, factorial of three.
- Three into factorial of two minus one I will do, then followed by two into factorial of the previous number.
- I've kept everything ready so that one second I will.
- Any time in this fine.
- Let me change the number.
- Factorial of five.
- And the expected answer is 120.
- And if you can see this is a name of a function, and this function factorial is again calling itself.
- This is fine.
- And I call this as a base case because when I say x becomes equal to 0, what it has to do?
- It has to return one.
- To whom it has to return?
- It has to return to a calling function.
- and we'll use a Python tutor.
- And then you have to say it as visualize the execution.
- Hope you have already learned this.
- Just go through this.
- And if you see the number of lines of code that I have written is 1, 2, 3, 4, 5, and 6.
- And how many number of steps that are involved here?
- 26 different steps are involved.
- Involved in order to execute the factorial program.

- That is what I said.
- It takes the amount of time it takes in order to execute is more.
- At the same time, internally, so many states have to be stored.
- Factorial function because if you can if you had seen your cursor, let me show this.
- The cursor is pointing at the beginning, the function header, okay?
- It basically processed the function header.

Example or application from the lecture

- So let me take up the same example.
- Let's quickly execute this program and find out what is the answer for this.
- Let's find out are there any errors or there are no errors, we'll find out.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000006 · EV0000007 · EV0000008 · +1 source units in pack.json

Core explanation · 10:00–14:59

I will get into the function body only when a particular function is called.

Key details

- And I'm now I'm trying to call what?
- Factorial of five.
- No, it is not equal to zero.
- If it is not equal to zero, what do I need to do?
- Go to your else part.
- What is x?
- X is 5, factorial of x minus 1 would be the previous number.
- 4 my sorry 5 minus 1, which is going to be 4.
- So go ahead with the next one and see.
- X becomes three.
- X becomes two.
- X becomes 1.
- The next time your X becomes 0.
- The moment your X becomes a 0, what will happen?
- It returns 1.
- It returns 1 to whom?
- To the one that has called.
- 2 into 3 was 6.
- 24 into 5 would become 120.
- I will tell you any number of programs that you do until and unless you code the problems.
- writing a recursive programs would be definitely tougher.
- Hope you have understood the basic concept and in the actual classes we will try to work with a few other programs.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000010 · EV0000011 · EV0000012 · +1 source units in pack.json

3 Callback

Study focus: Callback. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000014 · EV0000015

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

I am Nita Anne Jacob, Assistant Professor from the Department of Computer Science and Engineering.

Key details

- In this video, we're going to continue our discussion with functions, and what we will explore is the concept of callback.
- We know that you can pass any arguments to functions as input.
- But when a function is passed to another function as an argument, which is in turn invoked inside the outer function to complete a specific task, it is known as a function callback.
- Show you how exactly it works and what the concept really is.
- If I want to change this to F two, it's going to say.
- And the input to the interface is just one parameter which I'm just going to name as Fn the leader and internally and the body of the function, I'm going to just say fn and a function call.
- Whatever parameter or argument you pass here, it's going to be invoked as a function, which means the value of fn must be a function object.
- The function objects I have are F1 and F2.
- The functions F1 and F2 are called through the interface.
- Let me rephrase that.
- Fn is a function object that calls back the functions F1 or F2.
- I'm not going to change F1 or F2 here, but rather just pass the name of the function object.
- I'm going to clear the screen so you can see the error clearly.
- It says int object is not callable, which means in callback, the parameter.
- must receive a function object.
- A calling function can call the callback function as many times required, and all you have to do is just call the callback function once.

Example or application from the lecture

- So let me demonstrate this with an example to help you understand it better.
- And if I were to invoke F1, okay, let's go over to the interpreter, type Python.

- Let's look at the command prompt.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000014 · EV0000015 · EV0000016 · +1 source units in pack.json

Core explanation · 05:00–09:59

Within the interface function, I could have put a loop of say calling it four times, right?

Key details

- We're trying to just show the user what exactly the user needs to see.
- He doesn't need to know what are the intricacies of the application.
- Sorry, to destination.
- just say printing booked in Uber.
- I'm going to repeat the same task for Ola and create another function, Ola.
- If you open your Google Maps application and type a particular destination, apart from just the directions and how much time it takes from point A to point B, via car, bus, bike, walking, it also gives you an option to hire a cab.
- Why not exiting from Google and then going to Ola or Uber?
- When I invok app, it must get source and destination as the parameters as well.
- Google Maps function.
- If the value in app is equal to Uber.
- You can do that when you try this out.
- I if it's Uber, what I'm going to do is call G maps with source, destination, and app.
- But what will the app be?
- It's going to be a string, so I'm going to pass the function object Uber.
- I've taken care of Uber.
- I have to talk about Ola instead.

Example or application from the lecture

- One such example is that of your Google Maps.
- So I'm going to write remove this entire code and emulate an example of how we would use callback in Google Maps.
- And let's say I want to just book a cab from some source to a destination.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000018 · EV0000019 · EV0000020 · +1 source units in pack.json

Core explanation · 10:00–14:59

input errors like say else if and then again an else enter a correct app I'm just going to assume that my user is going to use the correct input right in.

Key details

- Ideal in an ideal world, this is an incorrect form of programming where I'm not taking care of the mistakes a user would make.

- But this is just to illustrate the concept of callbacks.
- It's not else if you have to give LF app equal to hola.
- And valid application.
- Space Electronic City.
- via Uber in uppercase letters, it's going to say enter valid application because we've only taken care of Uber and Ola in terms of case sensitive strings.
- I hope you have understood the concept of callback.
- App will be replaced with Uber and you're saying Uber of Source and Destination.
- I hope you've understood this concept.
- Do try this entire program out on Python Tutor to understand it further.
- I hope you will practice this and also try all the worksheets that are available on Pesso Academy.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000022 · EV0000023 · EV0000024 · +1 source units in pack.json

4 Functional and object oriented programming

Study focus: Functional and object oriented programming. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000026 · EV0000027

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we are going to be looking into the concept of functional programming.

Key details

- At the same time we will also un try to understand how.
- Python provides the different functions in order to work with the functional programming.
- Above all this, I really want to thank Professor Anis Kumar for providing us a beautiful examples in order to understand the concept of functional programming in an easy.
- I want you people to let me know what exactly these functions are trying to do.
- Fine, and there goes a parameter here, and then C.
- Define a list, an empty list in this case.
- What is this X?
- I will let.
- I'll say result.appen.
- Hope the functionality of an append with respect to a list.
- I'll say l of i.
- And after doing a computation, return the result for me.
- Having said this, let me create.
- This a list C B E S University.

- our very own Python.
- You can also write anything of your choice.
- Maybe more also, but I'll stop at this particular point.
- F1 and I will pass A as an argument here.
- I'll capture the result and then print it.
- I hope by this time you should have known what I'm trying to do here.
- And if you can see the result is fourteen, six and sixteen.
- What exactly is this?
- If you can see count this three, four, five, six, seven, eight, nine, ten, 10 plus 4, 14.
- Let me create a function.
- In this case, my function will be F2.
- There's got to be a parameter here also, which is x.
- I will just copy the same thing.
- But what I'll do is I'll do some changes.
- Hope this is okay with you all.
- And let me comment this so that you will not have any confusions here.
- Yeah, here we go.
- STR dot upper.
- STR dot upper.
- I'll have to change this because this is function two.
- And let me execute.
- When I execute this program, if you can see that all the elements of a list have been converted to its upper case.
- The observation goes like this.
- The observation is some initialization has been done.
- I have initialized certain things.
- And we are traversing through all the elements of an I trouble in this case.
- Is this fine?
- We have gone through all the elements of a list.
- List is an I tribble for you.

Example or application from the lecture

- Before I start with actual functional programming, let us try to write very simple programs and then understand what those programs are doing.
- So let me do one more simple example for you.
- Now, in both the examples, in the example one, where we had the function f1, and in this case this is my function f2, there are few things that we need to observe.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000026 · EV0000027 · EV0000028 · +1 source units in pack.json

Core explanation · 05:00–09:59

We are trying to apply some function to this.

Key details

- We have applied some function.
- In this case, the result is collected again in terms of a list.
- And then followed by the list has been returned.
- We did some initialization.
- Some initialization have been done.
- I am walking through every element of an I trouble.
- I take every element from an I tribble and apply some operation.
- And result is collected again in terms of an I triple itself, but in this case it is a list and I will return the list.
- The same thing we are trying to do here also.
- Okay, so now, so now what we are going to be doing next.
- In this case, I will rewrite the same set of programs in a different manner.
- Just look at how I will write the program.
- I will use the same list.
- Fine, I'll copy paste it here.
- Let me remove this.
- Hope this is not confusing to you.
- Just give me a time.
- I will remove this.
- And I will comment it till here.
- I do not have anything else here.
- This is my list.
- I'll say A is equal to map.
- Map of what?
- Map of Len, comma, A.
- Hope this is fine.
- Let me see print A.
- It's just fine.
- What am I trying to do?
- I'm making use of a function.
- In this case, map is a built-in function.
- Please let me remind you, map is a built-in function.
- Map is a built-in function which takes a callable.
- Along with that, it also takes an eye trouble.
- And hope I am not confusing you.
- I have a map.
- For this map.
- What did I do?
- And then I am calling a particular function.
- What is this function?
- This function is a callable function.
- I can call it as and when it is required.
- Hope there are no more confusions.
- As I mentioned, map takes two arguments, a callable and an iterable.

- What am I trying to do?
- What am I trying to do?
- Before me explaining, let me get the output first and then I will try to explain it.
- And if you can see, you're getting some map objects.
- You're getting a map object.
- What I will do is I will just change something like this, I'll say list of this.
- Is this fine?

Example or application from the lecture

- In this example, at the same time in the previous example also we have done this.
- You can see the previous example, aren't we doing the same thing here?
- For this particular map, I have given an I trouble fine and I have called a particular function len in this case.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000030 · EV0000031 · EV0000032 · +1 source units in pack.json

Core explanation · 10:00–14:59

That is what I'm trying to do.

Key details

- The reason is map is lazy.
- Map is lazy.
- Let me write map is lazy.
- Because of which what did I do?
- I said list of this.
- If you could not understand that particular statement, don't worry, you have an item, element, on this particular element, okay, on this particular element, what am I doing?
- I will apply this particular function.
- That's it.
- Nothing else we are trying to do.
- And one thing you should always remember when you work with your map function, when you work with your map function, the number of input elements will be equal to the number of output elements.
- That is the very important thing that you should always remember.
- Number of input elements will always be equal to the number of output elements.
- The one change is instead of len, I will say string dot upper.
- And that's it.
- Is this fine?
- You're saying take every element in an I tribble and apply a callable on it.
- That's what you are trying to do.
- Hope our pro problem is saved.
- Program is saved, and if you can see they've been converted into capital letters.
- Work out on the similar grounds so that you can understand the things in an easier manner.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000034 · EV0000035 · EV0000036 · +1 source units in pack.json

5 Lambda Function

Study focus: Lambda Function. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000038 · EV0000039

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we are going to be looking into the concept of a lambda.

Key details

- Lambda is basically a function without any name.
- There will not be any name for the functions.
- function without name.
- They're also called as they're famously called as anonymous functions.
- They're called as anonymous functions.
- They are also called as throw away functions.
- Lambda is not a statement, but it's an expression.
- And one important thing that you should always know with respect to lambda is lambda's body does not contain a block of statement.
- You will not have a block of statement.
- Instead, it's going to be a single expression.
- It's a single expression.
- I will basically use a lambda function when I do not want to use a function again and again.
- But in this particular case, if I make use of lambda, which means that I do not want to use this function again and again.
- And I will also make use of a lambda function whenever I need a function object.
- And a function object is required.
- the today when I use a lambda function, you may not appreciate the usage of the lambda functions.
- You want to use it for a shorter duration.
- The syntax for the lambda function is you will have lambda, it's a keyword.
- Lambda is a keyword which is used to define or declare an anonymous function.
- And I'll say either it you can have a single argument or you can have a number of arguments.
- This is fine.
- They all needs to be separated by a comma.
- Followed by a colon.
- A colon is a must.

- And then it's an expression.
- You will have an expression.
- And the results are returned back.
- Whenever you're working with a lambda function, you should never return a value.
- Do not return anything because it has an implicit written.
- If you cannot write a function using a lambda, then the regular way of writing you should always make use.

Example or application from the lecture

- Let's try to understand what exactly is a lambda, then we will also learn the usage of lambda.
- For example, in the previous sessions you have already seen how to create a function, and we have also given a name for a function, which means that I can reuse a function as and when it.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000038 · EV0000039 · EV0000040 · +1 source units in pack.json

Core explanation · 05:00–09:59

If your intention is to work with an inline function,.

Key details

- Fine, a single line of code, then always better to use a lambda function.
- X, Y, and Z is what I will try to write.
- And then I will say return.
- X plus Y plus Z.
- A simple one.
- X plus Y plus Z.
- This is what I'm trying to do.
- I will call the function.
- Two, three, and four.
- I'll say print key.
- I know it is the same program that you have already done.
- What we will try to do is we will try to convert this function into a lambda function.
- Let me convert this into a lambda function.
- A lambda as a keyword.
- You can have a single.
- If you have a single argument, you will specify it like this, then followed by a colon.
- And then followed by your expression.
- But in this case, if you can see that there are how many number of arguments are there?
- There are basically three arguments are there.
- And what are we trying to do?
- We are trying to perform addition in this case.
- This is what I will try to do.

- Two, three, comma four.
- This is how you try to write a lambda function.
- F is basically getting a function object.
- This function object is when a lambda expression creates it.
- Define also will work in a similar fashion, okay?
- DEF when we use it, it works in a similar fashion.
- But our assignment is going to happen automatically.
- In this case, the function object is available only when a lambda expression is created.
- Let me go here execute both of them.
- Hope the concept of lambda is clear to you.
- I'll write a program and I want you people to find out what I'm trying to do.
- And an argument is gonna be a single argument.
- I'll say A star A star A.
- I'm trying to find a cube of a particular number.
- That's it.
- Nothing else I will have to do.
- X of five.
- And you may also specify as it is a print statement Q value is Okay, so this is what something I will do.
- Function object X.
- X is a function object in this case.
- Sorry, let me execute this program.
- And C nine over the previous programs output because it's not been commented, and then followed by the Q value of five is what we were trying to find out.
- I request you to solve some of the other examples in order to understand the concept of lambda in a better way.

Example or application from the lecture

- And maybe we'll take up one more simple example and then we'll try to write.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000042 · EV0000043 · EV0000044 · +1 source units in pack.json

6 List Comprehension

Study focus: List Comprehension. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000046 · EV0000047

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Till now, we have understood how to create a list.

Key details

- A list can contain integers, it can be a float point, floating values, it can be strings, fine.
- In general, a list can be heterogeneous in nature.
- And this way of creating a list we call it as enumeration.
- And in today's class we will try to understand how to create list by using some rules.
- Creating a list using rules is called as list comprehension.
- There is also a dictionary comprehension, set comprehension, but they are all out of the scope for us so we are going to be learning only the concept of list comprehension.
- I in range.
- For I in range.
- while speaking I said it as five I in while I in range, but while typing I missed it.
- Can we really convert this into list comprehension?
- Can I write this using a rule?
- This is equal to.
- Because it's a list comprehension, I have to use a brackets here.
- Let me write for I in range.
- Now what is the next thing that I'll have to do?
- I'll have to print the values.
- Need what needs to be printed?
- It would have returned an object and I'll tell print this.
- The Python is printed for this.
- Let me comment on this.
- So now let me execute and no output because it has returned an object for us.
- We'll try to say print L1.
- Fine, and if you can see it has printed five times and within a list.
- And the general form.
- You have to write like this.
- General form is this.
- As usual, because it's a for loop variable.
- In eye trouble.
- for I variable in I triple.
- This is called as an expression.

Example or application from the lecture

- So just an example to show it to you, this is what we would have done.
- So let's execute this first and then we'll continue.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000046 · EV0000047 · EV0000048 · +1 source units in pack.json

Core explanation · 05:00–09:59

Execute the for a loop, fine, which is present as a part of list comprehension, execute this.

Key details

- Then every time you execute for loop, evaluate this expression.
- Execute for loops.
- Every time you execute a for loop, you will have to evaluate this particular expression.
- And thus the r resultant is a list that is getting created for us.
- This is an empty list, L1, that it's getting created at the beginning.
- I want to do square of a numbers.
- Square of numbers from.
- maybe one to ten?
- What do I need to do?
- Write a list comprehension, equivalentless comprehension.
- Because the moment I write level, it also includes a zero.
- And then do this.
- What I need to do?
- I need to perform the square of a number.
- And if you can see square of the numbers from one to ten has happened.
- I want to print.
- The string length.
- Along with that, I may also say Bangalore followed by the length.
- of strings and its length.
- [ASR low confidence; check audio].
- And I'll see PES University.
- Something of this kind.
- This is what I want to do and let me write a less comprehension.
- Or I in range.
- A for I in range.
- Sorry, not range.
- For I in A, because I already know I want to find out a string.
- and its length.
- It is available in I and I will say L of I.

Example or application from the lecture

- In the sense one to nine I want to do, do not consider ten.
- Or if you want to consider ten, you do it like this.
- Let's execute this and check.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000050 · EV0000051 · EV0000052 · +1 source units in pack.json

Core explanation · 10:00–14:59

Fine for I in a it should be fine.

Key details

- Execute this, I'm going to comment the next one also because I know what is the output here.

- Let me execute.
- Python six, PS University 14, language is eight, and so on.
- Please remember whenever you try to find the length, it also considers the space.
- I'll use the same list.
- And I want to convert all of them to uppercase.
- Fine, all of them to uppercase.
- I'll say L4 is equal to for INA because I'm still considering A itself.
- What do I need to do?
- I need to find the convert all of them to uppercase.
- You've already used this particular function in case of the strings request you to go back and check it out.
- There are so many things that you could do.
- You can also say that the one whose length is greater than six, print them and so on.
- I hope you have understood the concept of less comprehension.

Example or application from the lecture

- Let's use the same list.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000054 · EV0000055 · EV0000056 · +1 source units in pack.json

7 Closure

Study focus: Closure. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000058 · EV0000059

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Closure is applicable to nested functions.

Key details

- Nested functions have already been covered.
- Please look at the concept of nested functions once again.
- As I mentioned, the closure is applicable to nested functions.
- They can access the variables declared or the ones that are initialized inside this outer functions.
- Inner functions can access the variables that are initialized or declared within this outer functions.
- This accessing can be done even though the outer functions have been removed completely from the memory.

- The inner functions have an access to the variables declared or initialized inside an outer function even after the outer functions have been completely executed or it has been removed completely from the memory.
- The main intention is that the state information would be still retained even after the outer function has been removed completely from the memory.
- I will say it as an outer itself because I want to I should have an nested function.
- I'll say print some message.
- Print some message.
- address where it is present.
- I will say it as x is equal to outer.
- I will say I will have to pass some message.
- I'm passing some message.
- What should happen for me?
- I should be able to receive this.
- Is this fine?
- This is an outer enclosing function.
- And your inner is an nested function in this case.
- We're trying to print the message and return the inner.
- To which one it is going to be returned?
- It is going to be returned here, X.
- You are trying to call the innermost function.
- Hope it's not doing this.
- Give me time.
- We are doing this.
- See that was not saved.
- Let me check it once again.
- Sorry, sorry for the previous one.
- The program was not saved.
- Okay the last line when I typed it was not saved because of which you were not getting the output.
- What have we done?
- There are two functions, outer functions and inner functions.
- We have returned the inner functions address and I'm trying to call the function where it has been captured.

Example or application from the lecture

- Continue with one more example because when I was discussing I said.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000058 · EV0000059 · EV0000060 · +1 source units in pack.json

Core explanation · 05:00–09:59

Even after the outer function has been removed completely, I can still access the inner function.

Key details

- That itself is a concept of closure.
- This is just to explain you that the how inner functions and outer functions can be accessed.
- To help you out in this case.
- It is done in the similar manner.
- What do we do?
- We will do it in the similar fashion.
- I've called this particular function F1.
- At this point I will say delete f1.
- But what I will do is I will use this x and then call this function.
- I have defined a function f1.
- Nested function is F2.
- This nested function has a print statement.
- We are returning F2.
- The address of F2 is returned at this point.
- X contains the address of F2.
- I'm trying to delete F1.
- The moment I delete an outer function, what we basically thought is we cannot access an F2.
- But in general, it is not true.
- I can still access the.
- F2, the innermost function because it's a state information is stored in X.
- So that is why I call it as X, which means you're trying to call the innermost function which is F2.
- Let me comment on this.
- I'll say clear screen.
- Maybe you are getting confused with the previous output.
- I hope you have understood.
- In order to make things a little more clear to you, as usual, I will copy paste it onto the our Python tutor.
- The visualization tool that is there.
- Easy for you to understand.
- Number of steps involved is 11.
- Quickly look at what is happening.
- Headers are processed.
- Fine, only one header.
- The function has been called.
- The moment you call a function, you enter into the body of the function, fine, define F2.
- Again, when do you get into the F2?
- Only when F2 is being called.
- We haven't called F2 at this point.
- The moment you say return F2, to which one it is returning?
- From where you have been called.
- Just look at the visualization.
- F1 needs to be deleted.
- But if you can see parent f2's parent is still F1.

- And now you are calling F1.
- Try solving few more problems or programs on this in order to have a better understanding.
- Hope you have understood.

Example or application from the lecture

- Let's quickly try to do this.
- Let's go ahead with this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000062 · EV0000063 · EV0000064 · +1 source units in pack.json

8 Decorators

Study focus: Decorators. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000066 · EV0000067

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Python for computational problem solving.

Key details

- In today's class, we are going to be looking at the concept called as decorators.
- In order to understand decorators, there are a few other things that are required.
- And just to remind you, these things have already been the concepts that I'm going to be doing now have already been covered in the previous videos.
- I kindly request you to check.
- The ones that are required are.
- Will be executed by me now so that it is going to be easy for you to understand the concept of decorators.
- Let me create a function called as.
- And what I really want to do with this is to print this message.
- Let it be as simple as possible.
- I will also do something like this, F2 is equal to F1, right?
- The message that I'm trying to pass now is call as decorator.
- At this point.
- F1 and F2 are pointing to the same object internally.
- in order to make our life easy.
- So what is the next thing that I'm going to be doing is when we passed the arguments to a function, we passed some message and all.
- Function as an argument at this particular point.
- Let me define a function increment.

- Hope it is okay.
- So now in this case I will say a return x plus one because the incrementation operation and I will write yet another function called as decrement.
- And for the decrement also, I will pass the value, any value that of your choice.
- Just look at what I'm going to be doing now.
- Fine and I will retain its return value in this case.
- And then followed by this.
- This is fine for you.
- Hope there are no confusions here.
- This is clear?
- operations of let me call increment and the value that I will give is ten.

Example or application from the lecture

- Let's quickly execute this.
- Let's quickly execute this and check it out.
- Let's let me comment this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000066 · EV0000067 · EV0000068 · +1 source units in pack.json

Core explanation · 05:00–09:59

We quickly execute.

Key details

- And if you see there are no errors, and we are getting the result, which is 11.
- Body is processed, sorry, header is processed every time, sorry, it's a header.
- The function call is for this particular function.
- An increment is yet another function.
- Is this fine?
- And this x fine will get the value called as 10.
- Fine, because this function is increment and x value is 10.
- What it gets returned, it returns x plus 1.
- X plus 1.
- You are returning this particular result.
- You are returning it to this place where it was called.
- It has returned 11, and then you are trying to print a value called as 11.
- It's all about just understanding how things are going on, nothing much, just.
- Define one more function which is going to be called.
- Fine, a function called.
- Then followed by I will define one more function, function returned.
- Hope this is fine for you.
- Is it confusing?
- The question mark because I don't know.
- I'm unable to identify whether you're able to understand or not.

- And I will say return.
- Function underscore returned.
- Hope this is fine for you.
- I'll just come back here and then say something new is equal to the function that is called.
- What is the output that it gives us?
- And now I will say new of this.
- That's fine.
- These things have already been done up to you.
- You're calling a function, I get into the function.
- The moment the headers are processed, what is the next thing that I have to do?
- I'll have to wait for the function to be called.
- And then I will process this particular function.
- Will you get it into the body of this function return?
- No, I won't get it until it has been called.
- The object has been returned.
- Where it is returned, it is returned.
- To this particular point.
- It has returned the address where this particular function is available.
- Where this program basically illustrates that a function can return another function.
- Our actual concept is decorator.

Example or application from the lecture

- If there are any errors in this program, let's try to identify what is an error in this case.
- Let's find out.
- A function can return another function is what we have done or we have understood in this particular example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000070 · EV0000071 · EV0000072 · +1 source units in pack.json

Core explanation · 10:00–14:59

It takes one more function.

Key details

- Some other functionality.
- Let me repeat.
- It adds some other functionality.
- Hopefully it is clear and it reser returns the result.
- Decorator is a function, it takes another function, it adds some other functionality with the existing functionality and gives you the result.
- It is as similar as you wrapping a gift.
- You have a gift inside a box.
- What you basically do is you try to decorate your gift box in such a way that it is so nice.

- Whatever you try to do, you're trying to do it for your gift box and the object within that gift box remains the same.
- This is fine.
- Is this fine?
- You add some functionality to the existing code.
- The original code can be reused again and again according to your requirement because you are trying to use the same function, but add a few other things around it.
- That's called as a decorator.
- I say define make a pretty [ASR low confidence; check audio].
- [ASR low confidence; check audio].
- We do this.
- I look nice after decoration.
- I look nice after decoration.
- This is my function.
- I'm going to be calling this function.
- We'll see return function.
- This is not confusing you.
- I think then.
- What do I do?
- We'll try to call the function.
- Find out are there any errors in my program.
- Header gets processed, body never gets in, this header gets processed, but I am trying to call this function.
- Just to find out are there any errors in our program.
- And it says that I look very simple.
- Let me comment or let me remove this so that you will not have any confusions for you.
- So next thing that I'm going to be doing is pretty.

Example or application from the lecture

- Let me remind you, I'm taking the same example which is available on the internet itself because this example is very nice to understand.
- This example is from the internet material itself.
- Let me repeat, this is from the internet example itself.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000074 · EV0000075 · EV0000076 · +1 source units in pack.json

Core explanation · 15:00–19:59

And for this I possess an ordinary.

Key details

- I pass on ordinary.
- This is fine.
- Hope this is fine.

- Don't get confused, so I'll see pretty.
- Don't worry, I will show it on Python Tutor in order to understand.
- I look very simple now.
- You can change the statements.
- I look very simple now.
- After decoration, I look beautiful, all those things you can just do the changes and then work it out.
- What are we trying to do?
- We are trying to call make pretty.
- This is a place where decoration is taking place.
- Is this fine?
- This is a place where decoration is taking place.
- Make pretty, you're calling a function, and then you call ordinary.
- Is this fine?
- At this point, what does function contains?
- Your function contains ordinary.
- This is a point where your ordinary function has been called.
- Is this fine?
- Get a return address.
- The return address is present at pretty.
- I am calling the function.
- Pritty has inner function.
- The moment you call the inner function, what I'm doing trying to do, I look nice after decoration.
- When you and then.
- And followed by you have a function here.
- Previously I was looking very simple.
- After decoration that took place, I look beautiful.
- There is a few other things.
- We'll I'll just quickly do this.
- Let me eliminate these two sentences.
- I'm not going to be using this.
- I'll just call ordinary.
- This is fine.
- Nothing else I'm calling.
- The moment I call ordinary, it becomes like I look very simple.
- But now did I not eliminate the concept of decorator here?
- Yes, I have eliminated.
- An ordinary function pretty the make a pretty function.
- And I'm making this as what?
- A prettier one.
- This is fine.
- The last two statements that I eliminated, which is equal to these two.
- One second, give me time.

- Otherwise it just becomes an header processing without a function being called.
- And if you can see I look nice after decoration, I was looking very simple.
- I look very simple now.
- Previously, if you have seen our output remained the same.
- The last two statements I eliminated and this has been.
- Be placed with at the rate symbol followed by make written.
- Fourteen lines are there.
- Function was processed.
- Define the inner function.
- The moment you return, what are you trying to do?

Example or application from the lecture

- Let's quickly get started.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000078 · EV0000079 · EV0000080 · +1 source units in pack.json

Core explanation · 20:00–24:59

Where it has returned, it has returned to ordinary.

Key details

- Then now go back to where it has called.
- Then I look nice after decoration.
- And you're calling an ordinary function here.
- You come back here and then print I look very simple now.
- Hope you have understood.
- in the theory class when we do we will be looking into deeper into these things in order to understand this concept in a better way.
- Hope you are all fine over this.
- I request you to check it take out few more programs and work on the same.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000082 · EV0000083 · EV0000084 · +1 source units in pack.json

9 Generators

Study focus: Generators. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000086 · EV0000087

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we will understand the concepts of generators.

Key details

- Previous class previous class, we have learnt iTraders.
- In order to do this or implement our own iTraders, we had to use ITE.
- Okay and then followed by we were also forced to make use of a next method.
- I have to make use of next methods.
- and I have to keep track of the internal states.
- The reason is the moment I will end.
- All the values that are present in a particular iterable, I have to stop.
- Otherwise, what do I need to do?
- I need to explicitly raise a stop iteration in order to indicate that there are no more values that need to be returned.
- I have to implement a class, I have to make use of an iter, I have to make use of a method function, then take care of the count because the moment there are no more values to be returned I have to raise a stop iteration.
- we basically make use of a generators in order to build itrators in a simpler way.
- But all these things are automatically handled by the generators in.
- Let me write a simple function, define.
- I'll say it as what?
- Nothing more in this case.
- I'll just say it as N is equal to one.
- Let me write the program and then I'll explain it to you.
- This gets printed first.
- So this is the first statement that gets printed first.
- And I will use one new statement called as yield.
- And I'll say it as what yield one.
- That we want to implement generators, definitely.
- It is very easy because I'm going to be creating my function the way in which we have done now.
- Only difference is when I use my regular functions, I would use a statement called as return.
- Yield represents a generator, return represents a particular function.
- If you want.
- To create your own generators, then it should must and should contain at least one yield statement.
- If you find an yield statement, then it is easily noticeable that it is your generator function.
- Yield as well as return gives you some value from a function.
- The only difference is the moment you come across with a return statement.
- You're talking about termination of the function entirely.
- You come out of the function, okay, exit out of the function completely.
- But with respect to your yields, you're pausing the function.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000086 · EV0000087 · EV0000088 · +1 source units in pack.json

Core explanation · 05:00–09:59

You're going to be saving all its state and then continue its execution.

Key details

- Return is basically for termination.
- is to store internal storage, internally store the state where you were basically before.
- Let me change my statement and say second.
- Do this gives so.
- I'll say it as last.
- This gets printed last and then followed by eight n.
- I can do this.
- Print next of A.
- Ready with this.
- Python generators.py.
- See, this gets printed first and first statement is printed.
- One and two needs to be printed.
- pauses the function, saves its states, and later continues from where.
- there is a successive call continues from there to the successive calls.
- I will s the moment you come across with the yield function here, the state saving everything will happen.
- Till some point until it come across comes across us with the next yield statement.
- The moment you come across with the next yield, it returns a value for you.
- I'll start from here, go till here.
- Again I have an yield statement.
- You have to stop at this point.
- Okay, but now what is the point of me using a next anyway, fine?
- A is your generator object, and I'll say.
- A print A.
- For loop, right?
- I'm sorry.
- Okay, see here, this gets printed first, this gets printed second, this gets printed last.
- Okay, and I did not use any iter method or next method, but still we are able to create our own iterators using generators.
- Please try to understand the concept of classes and then come back and understand the concept of I traders.

Example or application from the lecture

- And then let's call our function a is equal to my generator.
- Suppose if I do this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000090 · EV0000091 · EV0000092 · +1 source units in pack.json

10 module

Study focus: module. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000094 · EV0000095

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Today we are going to explore one of the most powerful features of Python, which is modules.

Key details

- Modules help us write a clean, organized and reusable code and then play a major role in real-world software development.
- A module is simply a Python file that contains code functions, classes, or variables.
- That we can reuse them in other programs.
- Python already gives us many built-in modules like math.
- OS, random and date and time, etc.
- That makes our life easier.
- And the best part is we can create our own modules.
- We can create our own modules and even we have third party modules.
- What are the advantages of modules?
- Why do we use modules?
- First one is reusability.
- You write the code once and use it again and again.
- This saves a huge amount of time.
- Instead of writing code one, writing one long program, we break into smaller parts.
- Each part becomes easier to develop and debug.
- Next comes separate namespaces.
- Every module has its own space for names.
- Better organization, modul us structure a project neatly, especially when it grows larger.
- In short, modules makes our programs cleaner, smarter, and scalable.
- Next comes types of modules.
- We have three types built-in modules, user defined modules, and third party modules.
- Pip is a command to install packages.
- This is a tool to install and manage the third party Python packages.
- Then if you want to uninstall the module, you can give pip uninstall and the module name.
- And if you want to see what are the modules that you have installed.
- PIP list gives you a view of what all modules that you have installed.
- Almost every advanced Python project uses PIP.
- Next one is.
- We'll talk about the types of modules.
- Built-in modules.
- Built-in modules come packaged with Python.

- You don't need to install anything, just import them and use them.
- These modules for everyday programming needs.
- Next comes built-in modules, ways to import the built-in modules.
- and the module name.
- Import specific attributes.
- From math import square root.
- It is enough if you use square root, the just the function name.
- And in this case,.
- When you are importing specific attributes, you can only use those specific ones which you have imported, not all of them.
- Importing all from math import start.
- This is not recommended.
- If you don't if the specific attributes or specific functions, please import only them, not import star.
- Then next one is user defined modules.
- Whenever we write a Python file with some useful functions and save it as.py file, that can be imported to another program.
- This is an excellent way to organize your own function and reuse them across projects.

Example or application from the lecture

- Let's look at the advantages.
- Let's go deeper into each of these types.
- Before that, let's see what is pip.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000094 · EV0000095 · EV0000096 · +1 source units in pack.json

Core explanation · 05:00–09:59

Example files that you may create is you can create a.py file of any program and just you need to import use the inp keyword and import the program.

Key details

- It is the same like the built-in modules, but here you have created the own PY5.
- Then comes third party modules.
- Finally we have third party modules.
- These are created by Python community and gives us advanced capabilities.
- Some of the third party modules are listed down here.
- We have NumPy for numerical computing and we have Pandas for data analysis.
- or working with data sets.
- We have mapplotlib for charts and graphs, basically plotting and visualization.
- We have requests for fetching data from websites or working with APIs.
- We have pillow for image processing or image editing.
- These tools expand Python beyond basic programming and enable the real world application.

Example or application from the lecture

- Okay, example like NumPai, Skycatlone, Pillow, all of them has advanced capabilities.
- Built-in modules gives us ready-made functionality, user defined modules let us organize our own code, and third party modules bring the power of the entire Python community right into our projects.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000098 · EV0000099 · EV0000100 · +1 source units in pack.json

11 Numpy

Study focus: Numpy. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000102 · EV0000103

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Today we are going to explore NumPy, one of the most powerful libraries in Python, especially when it comes to scientific and numerical computing.

Key details

- Imagine you're standing in front of a huge mountain of numbers.
- Numbers from experiments, data, from marks of students or images filled with pixels.
- And Python looks at you and says, I can help, but not alone.
- This is exactly where NumPy enters a story.
- NumPy is like giving Python a superpower, the power to handle large numerical data elegantly, efficiently, and unbelievingly fast.
- NumPy, short for numerical Python, is not just a library, it's the foundation on which most data science and machine learning libraries are built.
- Its heart is the NB array.
- Think of it as a disciplined, highly optimized container that holds data in a clean and organized manner.
- Like a neat cupboard where every item has the same size, same type, and it is perfectly arranged.
- Let me tell you a short story.
- Imagine having a list of a million numbers and you want to multiply every value by two.
- If Python lists try to do this, they get tired.
- They walk slowly.
- They use a lot of memory on the way.
- But Numbai, Numbai does it in a single breath.
- Because the work is done deep inside optimized C code.

- Think of arrays as building blocks.
- You can have a 1D array like a simple list of marks.
- We usually call it as vectors.
- A 2D array like a table, we represent it as mattress, rows and columns, and then we have an N-dimensional.
- Array we usually represent call it as tensors, like stacks of images or lay of data.
- Number arrays keep everything clean, fast and mathematical.
- Creating arrays feels almost like magic spells.
- You can see the functions here, np.z, np.1s, and np.a range.
- NP dot zeros create a clean slate of mattress of zeros.
- Likewise, np dot one gives you a table full of ones useful for testing.
- NP.a range gives you numbers from zero to nine with a wave of your hand.
- With just few lines, massive data sets can be manageable.
- Imagine having two lists and wanting to add them.
- In pure Python, you would be probably using a loop.
- But Numai says why look?
- When I can do it instantly.
- And you have A multiplied by two and you get the multiplication of the A array.
- Number performs lightning fast operations on entire arrays.
- If you need to sort your data, NumPy gently rearranges it with the help of the function called sort.
- And you need to pass the array.
- If you need to join two arrays,.

Example or application from the lecture

- Today let's take a small and meaningful journey into this magical library called NumPy.
- Let's talk about the basics of arrays.
- Let's now see how to create arrays.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000102 · EV0000103 · EV0000104 · +1 source units in pack.json

Core explanation · 05:00–09:59

You can use concatenate function.

Key details

- It connects them smoothly.
- talking about reshaping arrays, reshaping feels like transforming clay into new shapes without changing the amount of the clay.
- Take a 1D array twist it and suddenly it becomes a two by three matrix using reshape.
- Reshaping is heavily used in images, ML models and simulations.
- It's not just technical, it's elegant.
- Then talking about the array attributes, every array quietly carries important information about itself.

- Size tells you the total elements, detail tells you what data lives inside.
- It's like introducing the array itself politely before you start working with it.
- Next comes indexing and slicing.
- Indexing in NumPy feels like reading exactly the part of the story you want.
- You want the fourth number, then you can give array of three because array indexes starts from zero.
- You want numbers from two to four, then you can use slicing.
- In a 2D matrix, matrix of one, two is like saying bring me the element in the second row and call third column.
- row 0, row 1 and row 2, column 0, column 1 and column 2, and we are getting this one.
- Array indexes starts from 0.
- So this is the first row and the second column.
- And add boolean indexing, that's your filter.
- It quietly handles you only the values that matter.
- What are aggregate functions?
- Think of aggregation functions as small detectives.
- They scan your data and report back.
- Max gives you the largest value.
- Min gives you the smallest value in the array.
- The average of the array.
- S T D talks about the standard deviation, the average, sorry, the spread or variation in the array.
- This gives you instant insights with almost no effort.
- Python lists and python arrays.
- Python lists are regular school bags, flexible.
- You can put many things but they are get heavy.
- But NumPy RAs are like professionally designed travel bag, organized, lightweight, optimised and ready for long journeys.
- When you deal with serious numerical data, you pick the travel bag.
- It teaches y us to think in arrays, to avoid loops, to embrace speed and to enjoy clean mathematical expression.

Example or application from the lecture

- Let's end with a simple comparison story.
- Here is given the difference between the lists and arrays.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000106 · EV0000107 · EV0000108 · +1 source units in pack.json

12 PIL

Study focus: PIL. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000110 · EV0000111

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Today we are going to explore Pillow, the modern version of Python Imaging Library, also called PIL.

Key details

- Simple and powerful library that helps us open images, edit images, and save images in multiple formats.
- It specially deals with image processing tasks.
- Pillow is widely used because it is easy to understand, support many image formats, and includes built-in functions for tasks like resizing, cropping, rotating and filtering images.
- To install Pillow module, we simply use Pip install pillow.
- To start with below, we first import the image class.
- Here's a basic code to open an image and display it on the screen.
- Image.show displays it using your system's default viewer.
- Pillow supports JPEG, PNG, BMP, JIF, TIF, and many more image formats.
- Once we load the image, we can check some important details like format, which tells the file type like JPEG, or GIF, or Tiv, etc.
- Then the size, which gives the width and height in pixels.
- Then the mode shows the colour representation such as RGB or grayscale.
- These properties help us understand the image before processing it.
- Resizing is one of the most common operations in image processing.
- Here's how we resize an image.
- But it may distort the aspect ratio.
- To maintain the aspect ratio we can use thumbnail.
- This ensures the image is resized without stretching or squashing.
- converting the image modes.
- You can use convert function to convert to an image to a different colour mode.
- The common colour modes are one, which is pure black and white, it's just one bit.
- L, which is grayscale.
- All the shades of gray colour will be represented in an 8-bit representation.
- Then RGB, red, green, blue colour, three channels, and this is also an 8-bit representation.
- Red, green, blue, and A stands for alpha, which is represents the transparency.
- Then C M Y K, which stands for Syan, Majenta, Low, and Black.
- Mostly used in printing.
- Mode conversion is especially useful in machine learning preprocessing, where special formats are required.

- To save an image that you have worked on, you can use save function.
- Then to rotate an image, you can use rotate function and give the degrees as argument.

Example or application from the lecture

- Let's begin.
- So this example is importing the image class and using the functions open and show.
- Next one, let's take a look into few more essential operations like saving images.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000110 · EV0000111 · EV0000112 · +1 source units in pack.json

Core explanation · 05:00–09:59

To crop an image, we can use crop function and the syntaxes you can give the pixel coordinates as left, upper, right, lower to extract the specific region from the image.

Key details

- These operations make Pillow extremely useful for automation workflows, image editing tasks, and pre processing pipelines.
- To summarize, Pillow is a very powerful and big enough friendly Python library for image processing.
- We learned how to use open images, display them, resize, convert color modes, rotate, crop and save images.
- With these foundations you can build a wide variety of image based applications from simple editors to machine learning pipelines.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000114 · EV0000115 · EV0000116 · +1 source units in pack.json

13 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000208 · EV0000209

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Functional Programming

- Functional programming is a programming paradigm where programs are
- The ability of functional programming languages to treat functions as values
- They depend only on the input given to them, and the output will be the

Slide 3 · Functional Programming

- It adopts lazy evaluation which avoids repeated evaluation because the value
- is evaluated and stored only when it is needed.
- Example: Functional Constructs like Lazy Map & Lists ,tuples, sets etc. 2

Slide 4 · The keyword - lambda

- Lambda functions are functions without the name.
- Lambda functions are used when function objects are required.
- Lambda functions are used when you need a particular function for a short

Slide 5 · The keyword - lambda

- Lambda functions in Python are very useful in defining the in-line functions.
- Most suitable when it is used as a part of another function(especially with map,

Slide 6 · The keyword - lambda

- lambda is a keyword used to declare lambda function.
- Input arguments: It can have multiple arguments (if there are more than
- one arguments, then separate the arguments by a comma).

Slide 7 · Example1: To find the square of a number (Comparison between

- def square(n): square=lambda n : n*n
- print(square(4)) print(square(4)) #Function Call

Slide 8 · Example 2: To find the greatest of two numbers using the relational

- maximum=lambdal bd a,bb : a iff a>bb elsel bb #max(a,b)(b) can alsol bbe usedd
- print("The maximum is ",maximum(6,7))
- <function <lambda> at 0x00000217A6F93550> <class 'function'>

Slide 9 · The power of lambda is better understood when you use them as an anonymous

- Suppose we have a function definition that takes one argument, and that
- argument will be multiplied with an unknown number.
- return lambda a : a * n

Slide 10 · We can use the same function definition to make two functions (doube_N and triple_N

- functions) in the same program.
- triple_N = myfunc(3) Output
- print(double_N(15)) 45

Slide 11 · Map() Function

- Python map() function is a built in function which is utilized to apply a function to each
- 2. The first argument for the map function should be a callable which takes one
- 3. This function will be applied to every element of the iterable, and creates a map

Slide 12 · Map() Function

- The map function causes iteration through the iterable, and applies the callable
- All these happen only conceptually as the map function is lazy and the above
- We can pass one or more iterable to the map() function.

Slide 13 · Map() Function

- Under lazy object creation, the object is created when it is needed whereas in the case of
- when that information is requested. Built-in functions like range(),map(),zip() and open()
- loop. In contrast, when we use list, which is an eager object, it basically computes all the

Slide 14 · Map() Function

- When deciding which evaluation strategy to use for a given programming problem,
- However, some general guidelines can be followed to help make an informed
- decision. For instance, lazy evaluation should be used when you want to delay or

Slide 15 · Map() Function

- Example 1 To double all numbers using map and lambda
- numbers = (1, 2, 3, 4)
- result = map(lambda x: x + x, numbers)

Slide 16 · Map() Function

- Example 2 – To add two lists using map and lambda
- result = map(lambda x, y: x + y, num1, num2)

Slide 17 · Map() Function

- Example 3 – Function that doubles even numbers present in the list.
- # Use map to apply the function to each element in the list
- result = list(map(double_num, numbers))

Slide 18 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000208 · EV0000209 · EV0000210 · +51 source units in pack.json

14 Source slide deck: UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator.

EV0000262 · EV0000263

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Functional Programming

- Functional programming is a programming paradigm where programs are
- The ability of functional programming languages to treat functions as values
- They depend only on the input given to them, and the output will be the

Slide 3 · Functional Programming

- It adopts lazy evaluation which avoids repeated evaluation because the value
- is evaluated and stored only when it is needed.
- Example: Functional Constructs like Lazy Map & Lists ,tuples, sets etc. 2

Slide 4 · The keyword - lambda

- Lambda functions are functions without the name.
- Lambda functions are used when function objects are required.
- Lambda functions are used when you need a particular function for a short

Slide 5 · The keyword - lambda

- Lambda functions in Python are very useful in defining the in-line functions.
- Most suitable when it is used as a part of another function(especially with map,

Slide 6 · The keyword - lambda

- lambda is a keyword used to declare lambda function.
- Input arguments: It can have multiple arguments (if there are more than
- one arguments, then separate the arguments by a comma).

Slide 7 · Example1: To find the square of a number (Comparison between

- def square(n): square=lambda n : n*n
- print(square(4)) print(square(4)) #Function Call

Slide 8 · Example 2: To find the greatest of two numbers using the relational

- maximum=lambdal bd a,bb : a iff a>bb elsel bb #max(a,b)(b) can alsol bbe usedd
- print("The maximum is ",maximum(6,7))
- <function <lambda> at 0x00000217A6F93550> <class 'function'>

Slide 9 · The power of lambda is better understood when you use them as an anonymous

- Suppose we have a function definition that takes one argument, and that
- argument will be multiplied with an unknown number.
- return lambda a : a * n

Slide 10 · We can use the same function definition to make two functions (doube_N and triple_N

- functions) in the same program.
- triple_N = myfunc(3) Output
- print(double_N(15)) 45

Slide 11 · Map() Function

- Python map() function is a built in function which is utilized to apply a function to each
- 2. The first argument for the map function should be a callable which takes one
- 3. This function will be applied to every element of the iterable, and creates a map

Slide 12 · Map() Function

- The map function causes iteration through the iterable, and applies the callable
- All these happen only conceptually as the map function is lazy and the above
- We can pass one or more iterable to the map() function.

Slide 13 · Map() Function

- Under lazy object creation, the object is created when it is needed whereas in the case of
- when that information is requested. Built-in functions like range(),map(),zip() and open()
- loop. In contrast, when we use list, which is an eager object, it basically computes all the

Slide 14 · Map() Function

- When deciding which evaluation strategy to use for a given programming problem,
- However, some general guidelines can be followed to help make an informed
- decision. For instance, lazy evaluation should be used when you want to delay or

Slide 15 · Map() Function

- Example 1 To double all numbers using map and lambda
- numbers = (1, 2, 3, 4)
- result = map(lambda x: x + x, numbers)

Slide 16 · Map() Function

- Example 2 – To add two lists using map and lambda
- result = map(lambda x, y: x + y, num1, num2)

Slide 17 · Map() Function

- Example 3 – Function that doubles even numbers present in the list.
- # Use map to apply the function to each element in the list
- result = list(map(double_num, numbers))

Slide 18 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000262 · EV0000263 · EV0000264 · +51 source units in pack.json

15 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000343 · EV0000344

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · reduce() Function

- The reduce() is a function that applies a given function to the elements of an iterable,
- reducing them to a single value. This function is defined in “functools” module.
- functools.reduce(function, iterable[, initializer])

Slide 3 · reduce() Function

- At first step, first two elements of the sequence are picked and the result is
- The•The samesame functionfunction isis appliedapplied totoo thethe previouslypreviously attainedattained resultresult andand thethe numbernumber justjust
- There will be n – 1 calls if no initializer is specified.(n is the number of elements in

Slide 4 · Example 1.To find the factorial of a number using reduce() function

- print("The factorial is ",functools.reduce(int.__mul__ , range(1, n + 1)))
- Example 2. To find the sum of first 10 numbers using reduce() function
- The sum of first 10 numbers is 55 10

Slide 5 · Example 3.To find the product of numbers with 100 as the initial value.

- print("The product with 100 as initial value
- is",functools.reduce(product, [11, 22, 33, 44], 100))
- The product with 100 as initial value is 35138400 11

Slide 6 · Example 4: To find the sum and maximum temperature

- print("The average temperature is ", sum/5)
- # using reduce to compute maximum temperature in the list
- print("The maximum temperature is : ", end="")

Slide 7 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000343 · EV0000344 · EV0000345 · +18 source units in pack.json

16 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000409 · EV0000410

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Zip() function

- Python zip() method takes iterable containers and returns a single
- The function zip is used to associate the corresponding elements of two
- It does not have any callback function.

Slide 3 · Zip() function

- The zip() function is used to combine two or more lists (or
- The resulting iterable contains tuples, where the first
- element from each list is paired together, and so on.

Slide 4 · Example 1: Consider the code below which pairs the two lists:

- for i in range(len(m)):
- The above can be done very easily and with less code using zip. We
- can observe the same output.

Slide 5 · Example 2: Combining two iterables (tuples) into a single iterable

- name = ["Sudha", "Suma", "Sara", "Asha"]
- roll_no = [404, 112, 393, 223]
- {('Sudha', 404), ('Suma', 112), ('Sara', 393), ('Asha', 223)}

Slide 6 · Example 3: Combining two iterables (lists) into a single list

- b = list(map(lambda x : x * x * x, a))
- print(list(zip(a, b)))
- [(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]

Slide 7 · Example 4: Zipping list and tuple with unequal size

- #Define lists for 'persons',and a tuple for 'ages'
- #lists along with the 'ages' tuple
- print("Zipped result as a list:")

Slide 8 · Example 5: Zipping list with unequal size.(Two ways)

- print(list(zip(lis1,lis2)))
- The same can be acheived using the directory comprehension as:
- print({k:v for k,v in (zip(lis1,lis2))})

Slide 9 · Practice programs

- 1.Given list of circle areas all in five decimal places, round each element
- in the list up to its position decimal places, meaning round up the first
- 3.The following are the scores of chemistry exam . Filter out those who

Slide 10 · Practice programs

- 4.You are given a list of fruits names. Using lambda and map function
- 6. Using the min and reduce function find the smallest number in a
- given list of 10 numbers.

Slide 11 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000409 · EV0000410 · EV0000411 · +30 source units in pack.json

17 Source slide deck: Max Min

Study focus: Source slide deck: Max Min. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000442 · EV0000443

SHORT-NOTE RECAP

Study notes · Source slide deck: Max Min

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · Solving- UE26CS151A

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 2 · max() and min() Functions in Python

- Python provides built-in functions to find the largest and smallest values among a
- set of numbers — no loops or manual comparisons needed.
- Great for finding the top score or • Great for finding the lowest score or

Slide 3 · max() Function – Introduction

- max() is a built-in Python function.
- It returns the largest value among the Syntax
- On a tie, it returns the first maximum

Slide 4 · max() Function – Example

- numbers = [10, 25, 15, 40, 5]
- max() scans every element in the list and returns 40, since it is greater than all the rest.

Slide 5 · min() Function – Introduction

- min() is a built-in Python function.
- It can be used with lists, tuples, sets
- On a tie, it returns the first minimum

Slide 6 · min() Function – Example

- numbers = [10, 25, 15, 40, 5]
- min() scans every element in the list and returns 5, since it is smaller than all the rest.

Slide 7 · Using max() and min() Together

- numbers = [12, 45, 7, 23, 56, 9]
- Calling max() and min() on the same list gives both extremes in just two lines of code.

Slide 8 · Quick Comparison

- Finds the largest / maximum value Finds the smallest / minimum value
- Great for scores, prices, rankings Great for lowest cost, fastest time
- max(3.5, 2.1, 9.9) → 9.9

Slide 9 · Key Takeaways

- They are useful for basic data analysis and problem solving.
- They're often combined with loops to analyze real data.
- The next slide covers a few advanced tricks worth knowing.

Slide 10 · Advanced Features: key and default

- Both functions accept extra parameters that make them far more flexible than a
- key= takes a function applied per item • default= sets a fallback value to return
- max(words, key=len) → longest word • max([], default=0) → 0

Slide 11 · Practice Questions

- 2. Write one line of code to find the smallest number in marks = [67,
- 3. What will max("apple", "fig", "kiwi") return, and why?
- 4. Write a program that reads 5 numbers and prints both the largest

Slide 12 · Department of CSE (AIML)

- Dr Manjula K-manjulak@pes.edu
- Ack: Teaching Assistant - Janavi Agrawal

EV0000442 · EV0000443 · EV0000444 · +33 source units in pack.json

18 Source slide deck: Intro to Modules

Study focus: Source slide deck: Intro to Modules. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000601 · EV0000602

SHORT-NOTE RECAP

Study notes · Source slide deck: Intro to Modules

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · Python Modules and PIP

- Department of Computer Science and Engineering
- Prof. Kundhavai K R, CSE Department

Slide 2 · Introduction to Modules

- Python provides many built-in modules like math, os, random,
- You can also create your own modules by saving functions in a
- You can also include the third-party modules in the python code.

Slide 3 · Advantages of Modules

- Reusability: Write code once and reuse it across multiple programs, reducing duplication and saving development time.
- Modularity: Break large programs into smaller, logical parts, making code easier to develop, debug, and maintain.
- Separate Namespace: Each module has its own namespace, preventing naming conflicts between functions or variables in different modules.

Slide 4 · Types of Modules

- User-Defined Modules (Creating your own modules).

Slide 5 · What is PIP?

- PIP = Pip Installs Packages Tool to install/manage third-party python packages.
- pip install package_name pip uninstall package_name pip list

Slide 6 · Types of Modules

- Provided by Python by default (e.g., math, os, random, datetime) No installation needed Useful for common programming tasks

Slide 7 · Types of Modules

- Ways to import Built-in Modules

Slide 8 · Types of Modules

- Modules created by the programmer Stored as .py files containing functions, classes, or variables Used to organize and reuse your own code

Slide 9 · Types of Modules

- my_utils.py → your helper functions calculator.py → custom math operations string_ops.py → string-related utilities

Slide 10 · Types of Modules

- Developed by the Python community
- Must be installed using PIP (e.g., numpy, scikit-learn, pillow)
- Extend Python with advanced or specialized features

Slide 11 · Types of Modules

- Third-Party Modules (via PIP)
- numpy → numerical computing pandas → data analysis matplotlib → plotting & visualization requests → working with APIs Pillow → image processing

Slide 12 · Department of Computer Science and Engineering

- Prof. Kundhavai K R, CSE Department

EV0000601 · EV0000602 · EV0000603 · +33 source units in pack.json

19 Source slide deck: Matplotlib

Study focus: Source slide deck: Matplotlib. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000676 · EV0000677

SHORT-NOTE RECAP

Study notes · Source slide deck: Matplotlib

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · Numerical Computing & Data Visualization in Python

- Professor, PCPS Anchor 2026 | Department of CSE (AIML)

Slide 2 · COURSE OVERVIEW

- 1 Visual cognition, Pyplot interface, and Figure vs Axes architecture.
- 2 Continuous trends with plt.plot(), discrete points with plt.scatter(), and bar/histograms.
- 3 Adding titles, units, grids, legends, and multi-panel figures with plt.subplots().

Slide 3 · 1. INTRODUCTION

- Rapid Cognition: Humans process images • Pyplot Interface: State-based module offering
- 60,000x faster than raw tables. MATLAB-like functions.
- Pattern Discovery: Instantly detect trends, cycles, • Standard Import: import matplotlib.pyplot as plt

Slide 4 · 1. INTRODUCTION

- Figure: The top-level canvas window holding all
- Axis: Individual X and Y lines determining scale,
- Artists: Every visible element (lines, text, dots) is

Slide 5 · 2. BASIC PLOTTING

- Basic Plotting: Line Plots (plt.plot)
- import matplotlib.pyplot as plt • color: 'b', 'r', or hex codes like '#1B365D' import numpy as np
- x = np.linspace(0, 10, 15) • linewidth (lw): Line thickness in points

Slide 6 · 2. BASIC PLOTTING

- Basic Plotting: Scatter Plots (plt.scatter)
- When to Use Scatter Plots?
- y = [5, 8, 10, 20, 15, 16, 32, 28] • Multidimensional: Encode 4 variables at once (X,

Slide 7 · 2. BASIC PLOTTING

- subjects = ["Python", "Math", "AI/ML", "DS", "COA"]
- plt.bar(subjects, marks, color='#1B365D', width=0.55)
- scores = np.random.normal(loc=74, scale=9, size=120)

Slide 8 · 3. CUSTOMIZATION

- CODE: Multi-Series Customization Essential Customization API

- x = np.linspace(0, 2 * np.pi, 100) • plt.legend(loc='best'): Identifies multiple datasets. # Plot multiple curves on same axes
- plt.plot(x, np.sin(x), color='#1B365D', • plt.grid(True, linestyle=':'): Adds reference

Slide 9 · 3. CUSTOMIZATION

- import numpy as np • fig, axes = plt.subplots(r, c): Multi-axis layout.
- plt.tight_layout(): Auto-adjusts padding between
- # Create 1 row, 2 columns of subplots • plt.savefig('file.png', dpi=300): Publication-ready

Slide 10 · 4. NUMPY + MATPLOTLIB

- Putting It Together: NumPy + Matplotlib
- CODE: NumPy Array Plotting (Sine & Cosine Example)
- # 3. Pass NumPy arrays directly to Matplotlib

Slide 11 · 5. SUMMARY

- NumPy & Matplotlib | UE26CS151A | Department of CSE (AIML) 11

Slide 12 · 5. SUMMARY

- NumPy & Matplotlib | UE26CS151A | Department of CSE (AIML) 1

Slide 13 · Department of CSE (AIML)

- Dr Manjula K — manjulak@pes.edu
- Ack: Mohammed Faizan – Teaching Assistant (CSE AIML)

EV0000676 · EV0000677 · EV0000678 · +36 source units in pack.json

20 Source slide deck: argparse

Study focus: Source slide deck: argparse. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000781 · EV0000782

SHORT-NOTE RECAP

Study notes · Source slide deck: argparse

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · argparse in Python

- Command-Line Argument Parsing: Syntax, Workflows & Applications
- Professor, PCPS Anchor 2026 | Department of CSE (AIML)

Slide 2 · COURSE OVERVIEW

- 1 Standard library module, eliminating hardcoded inputs, and automated --help manuals.
- The 4-Step argparse Workflow & Architecture
- 4 Type conversion (int/float), default fallbacks, choices whitelisting, and required flags.

Slide 3 · 1. INTRODUCTION

- for processing command-line arguments same script across multiple datasets and settings
- pass values directly into Python scripts from the containers, and CI/CD pipelines.
- descriptions when invoked with -h or --help.

Slide 4 · 2. CORE WORKFLOW

- Execution Pipeline Flowchart Step-by-Step Dataflow
- app.py --flag value. The operating system shell passes
- verified input values. If input is invalid, a formatted error

Slide 5 · 2. CORE WORKFLOW

- in argument parsing module without external # Step 1: Create the ArgumentParser object
- # Step 2: Register expected arguments
- # Step 4: Access and use parsed values and builds the args object.

Slide 6 · 3. ARGUMENT TYPES

- CODE: Positional Argument Example Key Rules: Positional Arguments
-) • Mandatory by Default: If omitted by the
- print(f"Square of {args.number} = with a missing argument error message.

Slide 7 · 3. ARGUMENT TYPES

- CODE: Optional Argument Example Key Rules: Optional Arguments
- # Optional argument with '--' and short alias '- be passed in any order anywhere on the
- print("Square =", args.number ** 2) define a fallback value when the user omits

Slide 8 · 4. PARAMETERS & VALIDATION

- default Fallback value default=10, Supplies default value if argument is omitted
- help Help documentation help="Path to input Text displayed next to argument in auto
- action Boolean flag control action="store_true" Sets attribute to True when flag is present, False

Slide 9 · 4. PARAMETERS & VALIDATION

- description="Display student record details" -> Name: Ananya | Age: 20 | Course: AI
- # Optional argument with default fallback -> error: the following arguments are required: --age
- default="Python", • Type Validation Error: $ python student.py --name

Slide 10 · 4. PARAMETERS & VALIDATION

- TERMINAL: Auto-Generated Help Output Built-In Documentation Features
- usage: student.py [-h] --name NAME --age the box without manual coding.
- CLI layout by running python script.py --help # structured and presented to the end

Slide 11 · 4. PARAMETERS & VALIDATION

- Validation with type and choices
- CODE: calc_validation.py Validation in Action
- description="Validated Arithmetic Tool" -> Op: add, Value: 10.5 (Processed successfully)

Slide 12 · 5. APPLICATIONS

- parser.add_argument("--data", required=True, • Shell Script Execution: $ python train.py
- help="Path to training CSV") --data dataset.csv --epochs 50 --lr 0.01
- default=0.001, configurations when submitting jobs to

Slide 13 · 5. APPLICATIONS

- Applications of argparse Across Industry
- connections, chunk sizes, and export batch sizes, learning rates, GPU device IDs,
- integration time steps, and tolerance values files in batches with customizable filtering

Slide 14 · 6. BEST PRACTICES & SUMMARY

- Best Practices for CLI Design
- input_file, --learning_rate instead of -i, -lr). units and expected formats in help='...'.
- type=int, type=float, or custom parsers to make flags required when the script

Slide 15 · 6. BEST PRACTICES & SUMMARY

- using Python's argparse library. Expected Output: Result = 15.0
- num2 as floating-point values (type=float, $ python calculator.py --num1 12.5 --num2 4
- with allowed choices: add, subtract, # Test Case 3: Invalid Choice Error

Slide 16 · 6. BEST PRACTICES & SUMMARY

- Core Technical Summary Key Takeaways for Students
- Python's built-in module for command-line hardcoded scripts into dynamic, reusable
- The 4-Step Pipeline: 1. Import -> 2. • Error Resilient: Catches missing

Slide 17 · Department of CSE (AIML)

- Dr Manjula K — manjulak@pes.edu
- Ack: Mohammed Faizan – Teaching Assistant (CSE AIML)

EV0000781 · EV0000782 · EV0000783 · +48 source units in pack.json

21 Historical question-source pages · unit mapping candidates

Study focus: Historical question-source pages · unit mapping candidates. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000832 · EV0000833

SHORT-NOTE RECAP

Study notes · Historical question-source pages · unit mapping candidates

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Archive page 5 from July__17cJZWr8dZ.pdf; unit match candidate score 0.75; source text/OCR may be partial. (4.0 Marks) (6.0 Marks) (5.0 Marks) (6.0 Marks) 4.b. Given the below mentioned list. Write a python code using lambda list1 = ['Java', 'Python', 'Csharp', 'Javascript', 'Go', 'Swift'] i) To convert the strings to lowercase if the length of it less than 4 and display it. ii) Display all the strings in the list which has the letter 'o' in it. 4.c. Explain the following module attributes. 1. __name__ 2. __doc__ 3. __dict__ 4.d. Write a function to mimic filter() - called myfilter(). Test this with the following. i)Given a list of strings, display all words whose length exceeds 6. 5.a. Explain the following with an example code. 1. Constructor 2. Inheritance 1 name of the module 2 has docstring 3 has all attributes(var,functions,modules) of class,object 1 used to initialize the attributes of a class when an object is created __init__ 2 repeat

Archive page 4 from May__1SdbvafL_8.pdf; unit match candidate score 0.4167; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 3.d. Explain the following functions with respect to the tkinter module in python with an example code. Tk() mainloop() 4.a. Mention any two ways of importing functions from a module. Explain one way with python code. 4.b. 1) Given a list, create a new list by removing negative numbers.
numbers = [1, -2, 3, -4, 5, -6] 2) Given a list having numbers in terms of strings, nums_strs = ["4", "8", "6", "5", "3", "2", "8", "9", "2", "5"], create a new list having the integers. 4.c. Given list1 = [101,3,3234,2331,31670,10], create a list of tuples using list comprehension where each tuple having an element from the given list and its length. initialise the main window of tkinter mainloop keeps the onscreen window to keep displaying in a constant loop unless closed by user import module module.function() from module import func1, fun2 func1()

Archive page 5 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.4167; source text/OCR may be partial. (4.0 Marks) (6.0 Marks) (5.0 Marks) (6.0 Marks) 4.b. Given the below mentioned list. Write a python code using lambda list1 = ['Java', 'Python', 'Csharp', 'Javascript', 'Go', 'Swift'] i) To convert the strings to lowercase if the length of it less than 4 and display it. ii) Display all the strings in the list which has the letter 'o' in it. 4.c. Explain the following module attributes. 1. __name__ 2. __doc__ 3. __dict__ 4.d. Write a function to mimic filter() - called myfilter(). Test this with the following. i)Given a list of strings, display all words whose length exceeds 6. 5.a. Explain the following with an example code. 1. Constructor 2. Inheritance

EV0000832 · EV0000833 · EV0000834

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.
Recursion transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Recursion transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Recursion transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Recursion transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/001_Recursion__57028129e466db02.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Callback transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Callback transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Callback transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Callback transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/003_Callback__da5d1859294b2983.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9986; only the JSON segmentation supplies the displayed timestamps.
Functional and object oriented programming transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/007_Functional_and_object_oriented_programming__8545ef40cb22fe08.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Lambda Function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lambda Function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lambda Function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9992; only the JSON segmentation supplies the displayed timestamps.
Lambda Function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
List Comprehension transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
List Comprehension transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
List Comprehension transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
List Comprehension transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/009_List_Comprehension__27fdb808f3d32a2b.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Closure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/010_Closure__193f0d4e9567775c.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Closure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/010_Closure__193f0d4e9567775c.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Closure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/010_Closure__193f0d4e9567775c.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Closure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/010_Closure__193f0d4e9567775c.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Decorators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Decorators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Decorators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Decorators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Generators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Generators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Generators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Generators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
module transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
module transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
module transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
module transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
PIL transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/015_PIL__b612657fcad391f3.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
PIL transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/015_PIL__b612657fcad391f3.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
PIL transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/015_PIL__b612657fcad391f3.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9991; only the JSON segmentation supplies the displayed timestamps.
PIL transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/015_PIL__b612657fcad391f3.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 1 2 Recursion	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__1__2_Recursion__163e7093cd196fcc.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 1 2 Recursion	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__1__2_Recursion__163e7093cd196fcc.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 1 2 Recursion (converted PDF; original hash 9420a0698c7b988bfbe4b209380d4b71f1ee5c8c566a505ecba3f200bc31627c)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__1__2_Recursion__163e7093cd196fcc.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 3 4 Callback	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/003_UE24CS151A_PCPS_Unit_3_Sindhu_Class__3__4_Callback__afde03d096f1327d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 3 4 Callback	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/003_UE24CS151A_PCPS_Unit_3_Sindhu_Class__3__4_Callback__afde03d096f1327d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 3 4 Callback (converted PDF; original hash d3ab1b6e2604ab559bbb7b935b2c18db5640c671de427ee1efc8a4d1c4d4459b)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/003_UE24CS151A_PCPS_Unit_3_Sindhu_Class__3__4_Callback__afde03d096f1327d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/005_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__2_Lambda__caf6259b1397b6cd.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/005_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__2_Lambda__caf6259b1397b6cd.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 2 Lambda (converted PDF; original hash c69fa1d290757f15ef100e1f1b00046aafaf411696deac13bc756b8db6abc0bf)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/005_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__2_Lambda__caf6259b1397b6cd.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/007_UE24CS151A_PCPS_Unit_4_Sindhu_Class__1__4_functional_programming_map_lambda__d6cfcde0f8bc4626.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/007_UE24CS151A_PCPS_Unit_4_Sindhu_Class__1__4_functional_programming_map_lambda__d6cfcde0f8bc4626.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 Sindhu Class 1 4 functional programming map lambda (converted PDF; original hash c69fa1d290757f15ef100e1f1b00046aafaf411696deac13bc756b8db6abc0bf)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/007_UE24CS151A_PCPS_Unit_4_Sindhu_Class__1__4_functional_programming_map_lambda__d6cfcde0f8bc4626.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 Sindhu Class 2 Functional programming filter	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/008_UE24CS151A_PCPS_Unit_4_Sindhu_Class__2_Functional_programming_filter__128520dfe54dc9f3.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 Sindhu Class 2 Functional programming filter	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/008_UE24CS151A_PCPS_Unit_4_Sindhu_Class__2_Functional_programming_filter__128520dfe54dc9f3.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 Sindhu Class 2 Functional programming filter (converted PDF; original hash b2b970153fac1bdc58b45e8e8c15d4d698bf046bc67cb73bf7cd52312c40f600)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/008_UE24CS151A_PCPS_Unit_4_Sindhu_Class__2_Functional_programming_filter__128520dfe54dc9f3.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__5_reduce__da5942d6e378890e.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__5_reduce__da5942d6e378890e.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 5 reduce (converted PDF; original hash cdb12182392eee5dc389bc82f688b599d1c50334adb30996d8fe15a0b455355c)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__5_reduce__da5942d6e378890e.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai class 6 list comprehension	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_class__6_list_comprehension__a50c71c4eae54791.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai class 6 list comprehension	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_class__6_list_comprehension__a50c71c4eae54791.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai class 6 list comprehension (converted PDF; original hash 2f022f752be4fbcd553f5659ab7e9a55c88a2ab601a93f1b41d07bf26e577441)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_class__6_list_comprehension__a50c71c4eae54791.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_class__6_zip__33aedb5fc4db15f6.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_class__6_zip__33aedb5fc4db15f6.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai class 6 zip (converted PDF; original hash 9ea9dfbdd9fe1b10a556e96effd277333e85906b56eaa6d30dbd724dfd0d310a)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/009_UE24CS151A_PCPS_Unit_4_SindhuRPai_class__6_zip__33aedb5fc4db15f6.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: Max Min	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/010_Max_Min__dc1351408347fdb8.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Max Min	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/010_Max_Min__dc1351408347fdb8.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Max Min (converted PDF; original hash 9030bbc43f626c9d4059b336d24f83cec3fde7a6a059583762d2af4bc06ffc16)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/010_Max_Min__dc1351408347fdb8.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 5 Closures	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/010_UE24CS151A_PCPS_Unit_3_Sindhu_Class__5_Closures__8889b68d5d038029.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 5 Closures	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/010_UE24CS151A_PCPS_Unit_3_Sindhu_Class__5_Closures__8889b68d5d038029.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 5 Closures (converted PDF; original hash b6d8cec7e1d295fa89e765fb77f7409d7e09b490bf4ba0fa58da3662e6700a18)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/010_UE24CS151A_PCPS_Unit_3_Sindhu_Class__5_Closures__8889b68d5d038029.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 6 Decorators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/011_UE24CS151A_PCPS_Unit_3_Sindhu_Class__6_Decorators__abed17696290de23.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 6 Decorators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/011_UE24CS151A_PCPS_Unit_3_Sindhu_Class__6_Decorators__abed17696290de23.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 6 Decorators (converted PDF; original hash 638ca53ee514b8782cf7ef31c74ac54db7900089d4ceb515be5745db37044774)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/011_UE24CS151A_PCPS_Unit_3_Sindhu_Class__6_Decorators__abed17696290de23.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 7 Generators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/012_UE24CS151A_PCPS_Unit_3_Sindhu_Class__7_Generators__6cae1ddd3709b089.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 7 Generators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/012_UE24CS151A_PCPS_Unit_3_Sindhu_Class__7_Generators__6cae1ddd3709b089.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 7 Generators (converted PDF; original hash 5cf73012838ce6448fb7d713f14ad27c99d12172327d48654f3766959df11a3a)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/012_UE24CS151A_PCPS_Unit_3_Sindhu_Class__7_Generators__6cae1ddd3709b089.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: Intro to Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/013_Intro_to_Modules__9bad64887fc1f1ee.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Intro to Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/013_Intro_to_Modules__9bad64887fc1f1ee.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Intro to Modules (converted PDF; original hash 3ee2df63ab4e17c6d560369f5eb890fe24dcf446b09e940e4d4e0bb7a64a938c)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/013_Intro_to_Modules__9bad64887fc1f1ee.pptx	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Numpy	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/014_Numpy__084bdca3b5d7a7d1.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Numpy	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/014_Numpy__084bdca3b5d7a7d1.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Numpy (converted PDF; original hash 1f45831a80858ac542eab36ae1f6314923a768837afc74abdb2920f0862c95c5)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/014_Numpy__084bdca3b5d7a7d1.pptx	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: Matplotlib	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/015_Matplotlib__26db950762ee81e0.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Matplotlib	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/015_Matplotlib__26db950762ee81e0.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Matplotlib (converted PDF; original hash 5cd46f020e8794e90586ff1945ceeb71d0a2877bac205bf8f679cf08d833925c)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/015_Matplotlib__26db950762ee81e0.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: PIL	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/015_PIL__5a5c2ba9aaa68100.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: PIL	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/015_PIL__5a5c2ba9aaa68100.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for PIL (converted PDF; original hash 9745d2e78ab3b998e455db5c82600e448708bce7c19b6d765270dbc9663fc345)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/015_PIL__5a5c2ba9aaa68100.pptx	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: Re Module	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/016_Re_Module__9097da6befb51bf8.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Re Module	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/016_Re_Module__9097da6befb51bf8.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Re Module (converted PDF; original hash ebeace81bbef2fa2539f3644e690a4ca793e495a51782decd5bd38e1b76a8b2f)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/016_Re_Module__9097da6befb51bf8.pptx	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: argparse	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/016_argparse__7901fba9314fe002.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: argparse	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/016_argparse__7901fba9314fe002.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for argparse (converted PDF; original hash 4bbb04f9d373166155a288e6defac4b1bb748b0d64c1ca351c00b27f9790911e)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/Slides/016_argparse__7901fba9314fe002.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Extracted historical pages: July__17cJZWr8dZ.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/July__17cJZWr8dZ.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Source	Path	Use
Extracted historical pages: May__1SdbvafL_8.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/May__1SdbvafL_8.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: July__1e0jrB6Yh8.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/July__1e0jrB6Yh8.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00001	1 covered · 0 excluded · 0 gaps	catalogue: Unit title and index copied from the official units.json catalogue; human content review is pending.

Source	Unit counts	Example decisions
SRC00002	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...
SRC00003	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00004	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...
SRC00005	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0050. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00006	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00007	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00008	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00009	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00010	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...
SRC00011	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00012	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...
SRC00013	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0053. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00014	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...
SRC00015	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...
SRC00016	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00017	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...
SRC00018	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00019	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00020	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00021	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0049. Sidecar variants compare...
SRC00022	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0038. Sidecar variants compare...
SRC00023	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0038. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00024	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0038. Sidecar variants compare...
SRC00025	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0038. Sidecar variants compare...
SRC00026	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00027	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...
SRC00028	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00029	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...
SRC00030	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...
SRC00031	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00032	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...
SRC00033	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...
SRC00034	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00035	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00036	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00037	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00038	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...
SRC00039	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...
SRC00040	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00041	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...
SRC00042	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0025. Sidecar variants compare...
SRC00043	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0025. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00044	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0025. Sidecar variants compare...
SRC00045	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0025. Sidecar variants compare...
SRC00046	0 covered · 0 excluded · 22 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00048	0 covered · 0 excluded · 22 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00047	0 covered · 0 excluded · 22 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00049	0 covered · 0 excluded · 8 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00051	0 covered · 0 excluded · 8 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00050	0 covered · 0 excluded · 8 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00052	18 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00054	18 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00053	18 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00055	18 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00057	18 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00056	18 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00058	0 covered · 0 excluded · 9 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00060	0 covered · 0 excluded · 9 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00059	0 covered · 0 excluded · 9 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00061	7 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00063	7 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00062	7 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00064	0 covered · 0 excluded · 15 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00066	0 covered · 0 excluded · 15 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00065	0 covered · 0 excluded · 15 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00067	11 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00069	11 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00068	11 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00070	12 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00072	12 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00071	12 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00073	0 covered · 0 excluded · 13 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00075	0 covered · 0 excluded · 13 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00074	0 covered · 0 excluded · 13 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00076	0 covered · 0 excluded · 16 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00078	0 covered · 0 excluded · 16 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00077	0 covered · 0 excluded · 16 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00079	0 covered · 0 excluded · 12 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00081	0 covered · 0 excluded · 12 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00080	0 covered · 0 excluded · 12 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00082	12 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00084	12 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00083	12 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00085	0 covered · 0 excluded · 13 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00087	0 covered · 0 excluded · 13 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00086	0 covered · 0 excluded · 13 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00088	13 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00090	13 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00089	13 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00091	0 covered · 0 excluded · 9 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00093	0 covered · 0 excluded · 9 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00092	0 covered · 0 excluded · 9 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00094	0 covered · 0 excluded · 13 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00096	0 covered · 0 excluded · 13 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00095	0 covered · 0 excluded · 13 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00097	17 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00099	17 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00098	17 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00100	1 covered · 0 excluded · 0 gaps	q5: Original archive PDF page 5; candidate unit association is text-based and unconfirmed.
SRC00101	1 covered · 0 excluded · 0 gaps	q4: Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
SRC00102	1 covered · 0 excluded · 0 gaps	q5: Original archive PDF page 5; candidate unit association is text-based and unconfirmed.

C Revision and retrieval practice